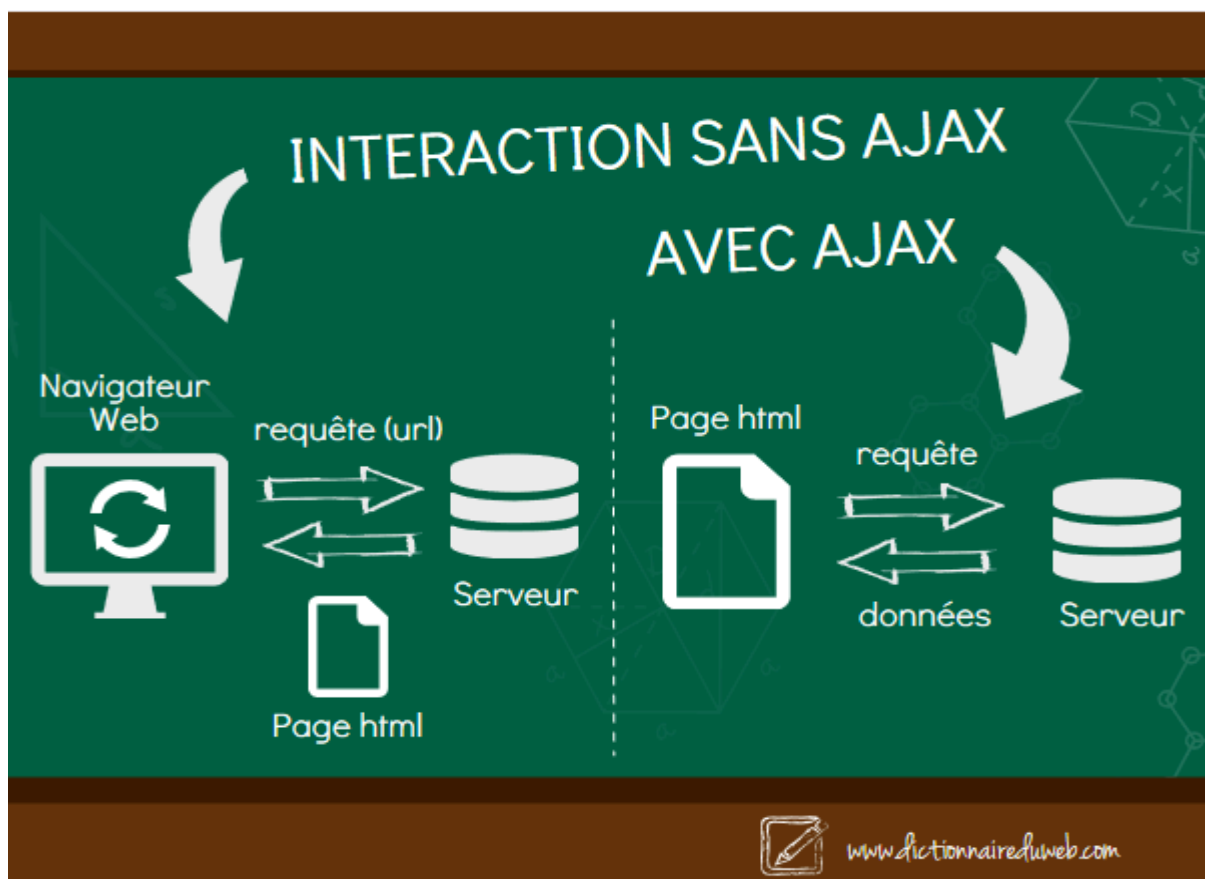


Cours

1.1 — Le problème : pourquoi AJAX ?

Dans une application web classique, chaque interaction qui nécessite des données fraîches du serveur (recherche, filtre, mise à jour...) déclenche un rechargement complet de la page. C'est lent, ça fait clignoter l'écran, et l'utilisateur perd sa position dans la page.

AJAX (Asynchronous JavaScript And XML) résout ce problème en permettant à JavaScript d'envoyer une requête HTTP vers le serveur en arrière-plan, sans interrompre l'affichage. La page reste visible et interactive pendant que la requête part. Quand la réponse arrive, JavaScript met à jour uniquement la zone concernée du DOM.



1.2 — Le flux d'une requête AJAX

Voici ce qui se passe entre la frappe de l'utilisateur et l'affichage des résultats :



1.3 — L'objet XMLHttpRequest

C'est l'objet JavaScript qui gère toute la communication. Il s'utilise toujours dans le même ordre :

```
// 1. Créer l'objet
const xhr = new XMLHttpRequest();

// 2. Configurer : méthode HTTP, URL, asynchrone (true)
xhr.open('GET', 'recherche.php?q=ga', true);

// 3. Définir ce qui se passe à chaque changement d'état
xhr.onreadystatechange = function() {

    // 4. Agir uniquement quand c'est terminé ET réussi
    if (xhr.readyState === 4 && xhr.status === 200) {

        // 5. Convertir la réponse texte en tableau JavaScript
        const personnes = JSON.parse(xhr.responseText);

        // 6. Mettre à jour l'interface
        afficherResultats(personnes);
    }
};

// 7. Envoyer la requête (toujours en dernier !)
xhr.send();
```

1.4 — readyState et status : à quoi servent les deux ?

<code>readyState</code>	0 → 1 → 2 → 3 → 4	État d'avancement de la requête (0 = non initialisée, 4 = terminée)
<code>readyState === 4</code>	4	La réponse est entièrement reçue — on peut la lire
<code>status</code>	200 / 404 / 500	Code HTTP retourné par le serveur (comme dans l'URL d'un navigateur)
<code>status === 200</code>	200	Le serveur a traité la requête avec succès

TP – Affichage des utilisateurs dynamique

Contexte

Vous allez reproduire le comportement vu sur le formulaire d'inscription universitaire: un champ de recherche qui affiche en temps réel une liste de personnes correspondant aux lettres saisies, sans recharger la page.

Le fichier [recherche.php](#) est fourni et fonctionnel : il contient 25 personnes codées en dur (pas de SQL) et retourne un tableau JSON filtré selon le paramètre `?q=`.
Votre unique travail : écrire le JavaScript.

Ce que doit faire l'application

1. L'utilisateur tape dans le champ de recherche
2. Dès 2 caractères saisis, la recherche se déclenche automatiquement
3. JavaScript envoie une requête AJAX à `recherche.php?q=SAISIE`
4. Les résultats s'affichent sous le champ, sous forme de liste de suggestions
5. Si moins de 2 caractères : la liste est vidée et masquée
6. En cas d'erreur serveur : afficher un message d'erreur

Les exercices:

1. Sélectionner les éléments du DOM

Récupérez `#champRecherche` (l'input) et `#liste-resultats` (le conteneur des suggestions) avec `getElementById`.

2. Écouter la frappe avec l'événement 'input'

Branchez un écouteur sur l'événement 'input' du champ. Cet événement se déclenche à chaque caractère tapé ou effacé. À chaque déclenchement, appelez `lancerRecherche()`.

3. Écrire `lancerRecherche()`

Lisez la valeur du champ (`.value.trim()`). Si elle fait moins de 2 caractères, appelez `viderListe()` et arrêtez-vous (`return`).

Sinon, construisez l'URL `'recherche.php?q=' + encodeURIComponent(saisie)`, créez un `XMLHttpRequest`, ouvrez-le en GET asynchrone, définissez `onreadystatechange` pour appeler `afficherResultats()` quand `readyState === 4` && `status === 200`, et envoyez avec `.send()`.

4. Écrire `afficherResultats(personnes)` et `viderListe()`

`viderListe()` : vide `#liste-resultats` et le masque (`style.display = 'none'`).

`afficherResultats(personnes)` : vide la liste, si le tableau est vide appelez `viderListe()`, sinon montrez le conteneur (`style.display = 'block'`) et créez pour chaque personne un `<div class="suggestion">` contenant le nom complet et le service, que vous injectez dans `#liste-resultats`.